YILDIZ TECHNICAL UNIVERSITY
FACULTY OF CHEMISTRY AND METALLURGY
DEPARTMENT OF MATHEMATICAL ENGINEERING

Module MTM4962 - Project Milestone Report

REAL ESTATE MANAGEMENT SYSTEM

Lecturer: Dr. Fettah Kıran

Student ID, Name, Surname
21052012 : Efe Kağan Önem
22058033 : Ece Naz Özbucak
22058053 : Suna Seyrek
21058072 : Zekiye Miray Karagöz

İstanbul, 2025

1. PROBLEM STATEMENT

The real estate industry is inherently data-intensive, requiring the management of complex interconnections between property specifications, ownership history, and shifting client preferences. In many agencies operating across multiple districts, this vital information is fragmented across disconnected spreadsheets and manual files. Such decentralization inevitably leads to operational failures, including contradictory property statuses, duplicated client records, and a lack of oversight regarding agent performance.

This project addresses these challenges by implementing a normalized relational database that centralizes all core operations into a single, consistent data source. The system moves beyond simple storage by modeling a realistic multi-office workflow where agents, seniority levels, and reporting lines are clearly defined. By tracking a property through its entire lifecycle from the initial listing through various viewings to the final closed transaction the database ensures that every piece of data is linked, accurate, and accessible across the entire organization.

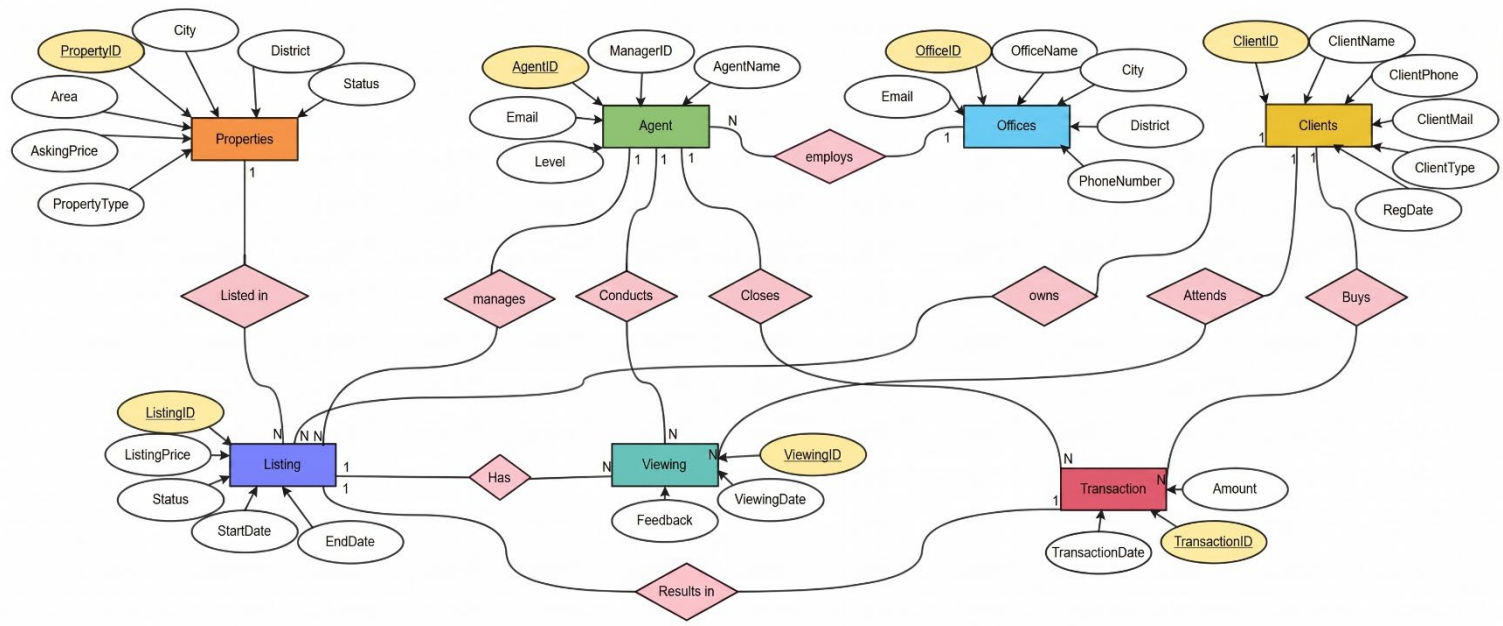
2. WHY IT MATTERS

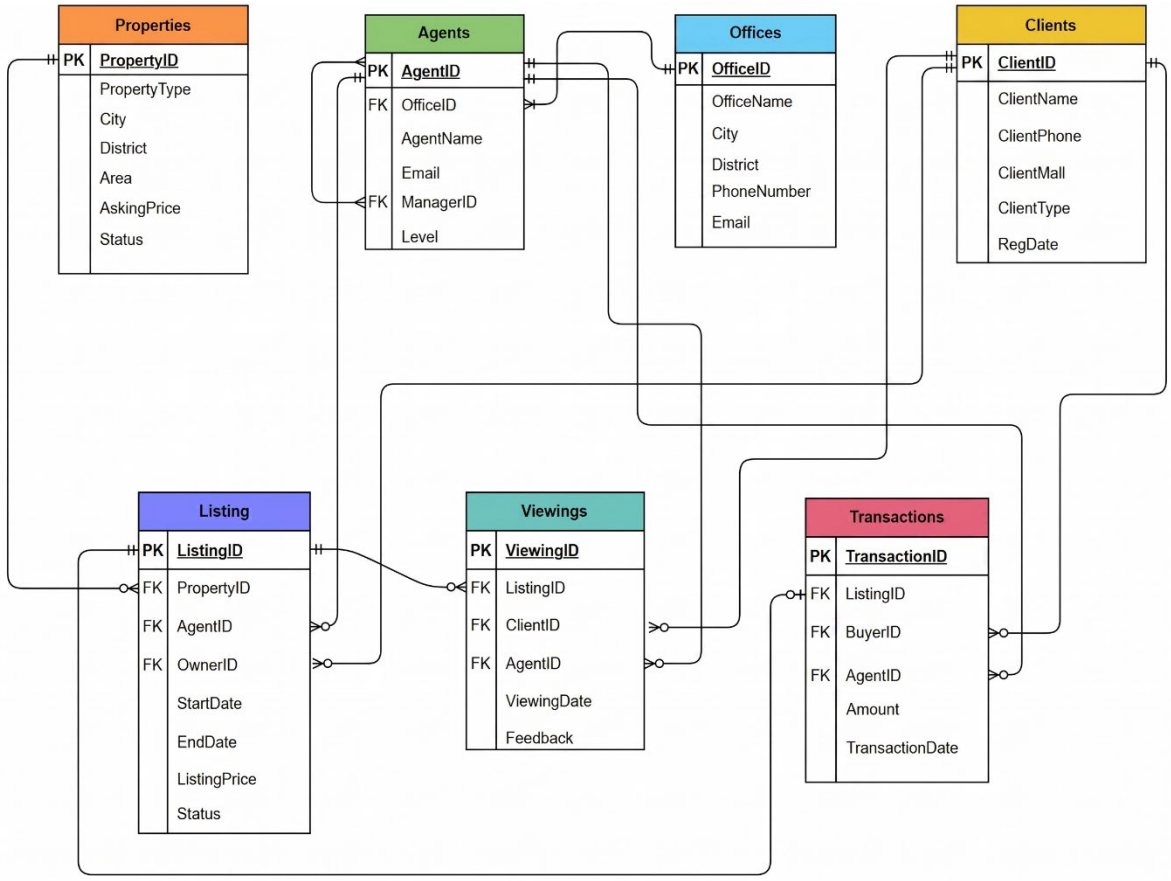
The real estate industry is highly competitive, and agencies that can operate efficiently with accurate, up-to-date information have a significant advantage over those relying on manual processes. A well-designed relational database transforms how an agency operates at every level from the individual agent scheduling a viewing to the regional manager reviewing monthly revenue.

A well-designed database for a real estate agency provides the following concrete business value:

- **Operational efficiency:** Agents can instantly query available properties that match a buyer's budget, preferred district, or property type eliminating the need to manually search through files or call colleagues in other offices.
- **Performance tracking:** Management can run aggregation queries to measure each agent's total closed deals, compare revenue across offices, identify which property types sell fastest, and spot agents who are generating many viewings but few closings a potential training signal.
- **Client relationship management:** The system maintains a complete profile for every client, including whether they are a buyer, seller, or both, when they registered, and their full viewing and transaction history. This supports better service and targeted follow-up.
- **Audit trail and compliance:** Every viewing and transaction is recorded with a date, responsible agent, and outcome. This provides a legally reliable history of all agency activity, which is important for dispute resolution and regulatory compliance.
- **Scalability:** The normalized schema is designed to grow cleanly. Adding a new city, opening a new office, or onboarding dozens of new agents and properties requires no structural changes only new rows in the appropriate tables.

3. PRELIMINARY SCHEMA





4. PROPOSED SQL FEATURES

Views

Views can be used to simplify recurring queries by combining multiple tables into a single queryable object. For example, a view could join Listings, Properties, Agents, and Offices to display all active listings in one place, or consolidate a client's viewing history and transactions together, removing the need to write complex joins repeatedly.

Joins

Joins are essential for retrieving meaningful data across the normalised schema. Inner joins could link Listings with Properties and Agents for standard reports, while left joins would allow all listings to be retrieved even if no viewings have been recorded yet. A self-join on the Agents table could pull each agent alongside their manager in a single row.

Aggregations

Aggregate functions could power a range of statistical reports, such as tracking how many viewings a listing has received, calculating total revenue per office, or averaging property prices by district. GROUP BY with HAVING could then filter these results to highlight agents or offices that meet certain performance thresholds.

CTEs (Common Table Expressions)

CTEs could simplify complex queries that would otherwise require nested subqueries. A standard CTE could rank agents by transaction volume within each office, while a recursive CTE could traverse the management hierarchy upward from any given agent through to senior management.

Subqueries

Subqueries could be used to answer specific business questions, such as identifying properties that received viewings but never led to a sale, or finding clients who appear as both a property owner and a buyer across the database.

Github Repo Link for the interested: https://github.com/kagannonem/SQL-Based-Real-Estate-Management-System